

Data Structures – Unit 06

Bucharest University of Economic Studies

Queues



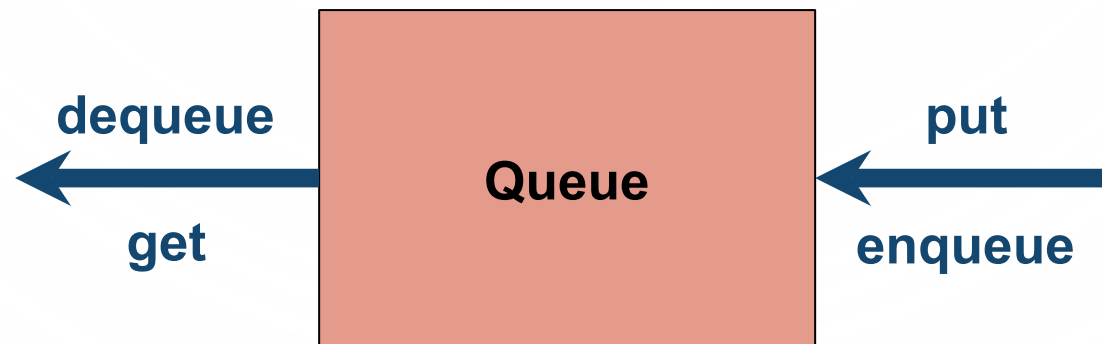
Agenda

- Definition
- Graphical representation
- Internal interpretation
- Characteristics
- Operations
- Implementations



QUEUES

It is a linear structure in which the two basic operations, insertion and deletion are made on a FIFO based model. The insertion is made on the tail section and the deletion is made on the front section of the queue.



Queue

- A logical structure implemented with different type of data structures: linked lists or arrays;
- A homogenous structure with elements of the same type;
- Basic operations:
 - Adding an element, called the enqueue or put operation
 - Removing an element, called the dequeue or get operation

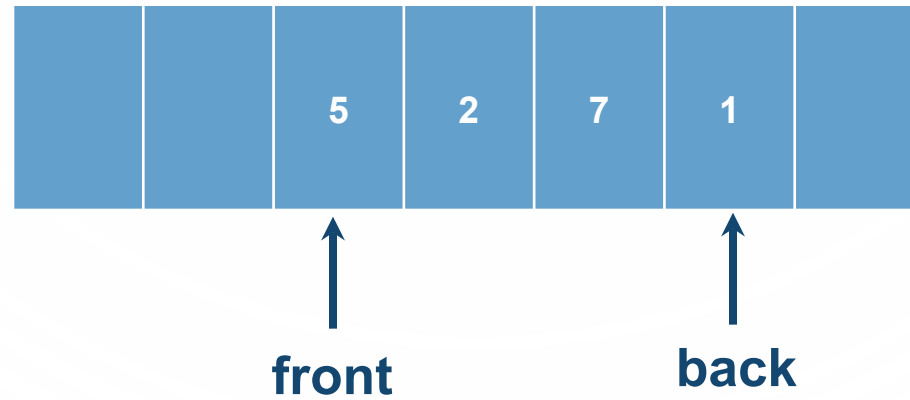
```
struct Queue{  
    Node* head;  
    Node* tail;  
};
```

```
struct Node{  
    int  
    Node *  
    info;  
    next;
```

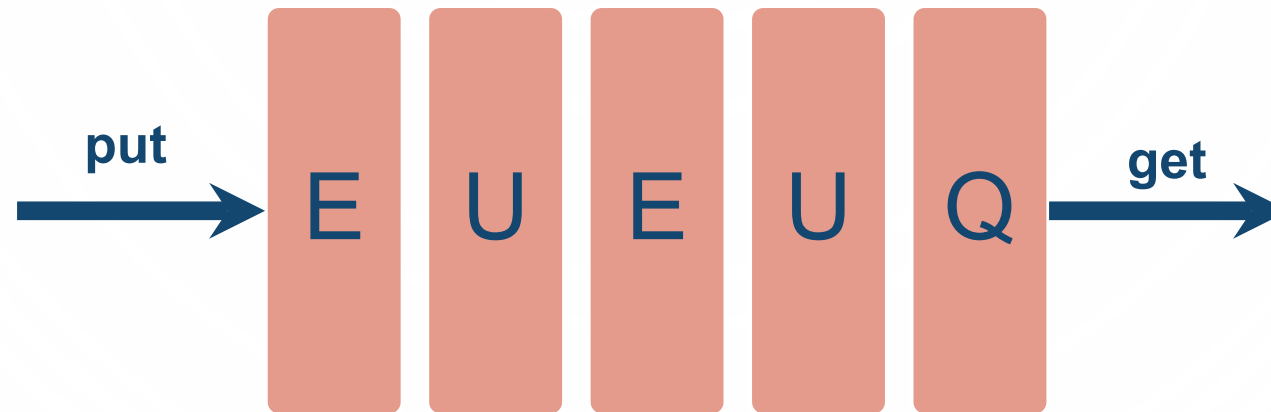


Queue

Implementation model
FIFO – **F**ifo **I**n **F**irst **O**ut



Queue – Implementation



PUT Pseudocode

1. memory allocation for a new node (newNode);
2. newNode initialization;
3. if (full queue)

queue overflow;

else

if (empty queue)

head = tail = newNode ;

else

{

tail->next = newNode;

tail = newNode;



GET Pseudocode

1. declaring a temporary node (tmpNode);
2. if (empty queue)

queue underflow;

else

value = head->info;

tmpNode = head;

head = head->next;

free(tmpNode);

return value;



Queues with Linked Lists

head of the queue

2	1005
1002	1003

5	1328
1005	1006

2	NULL
3241	3242

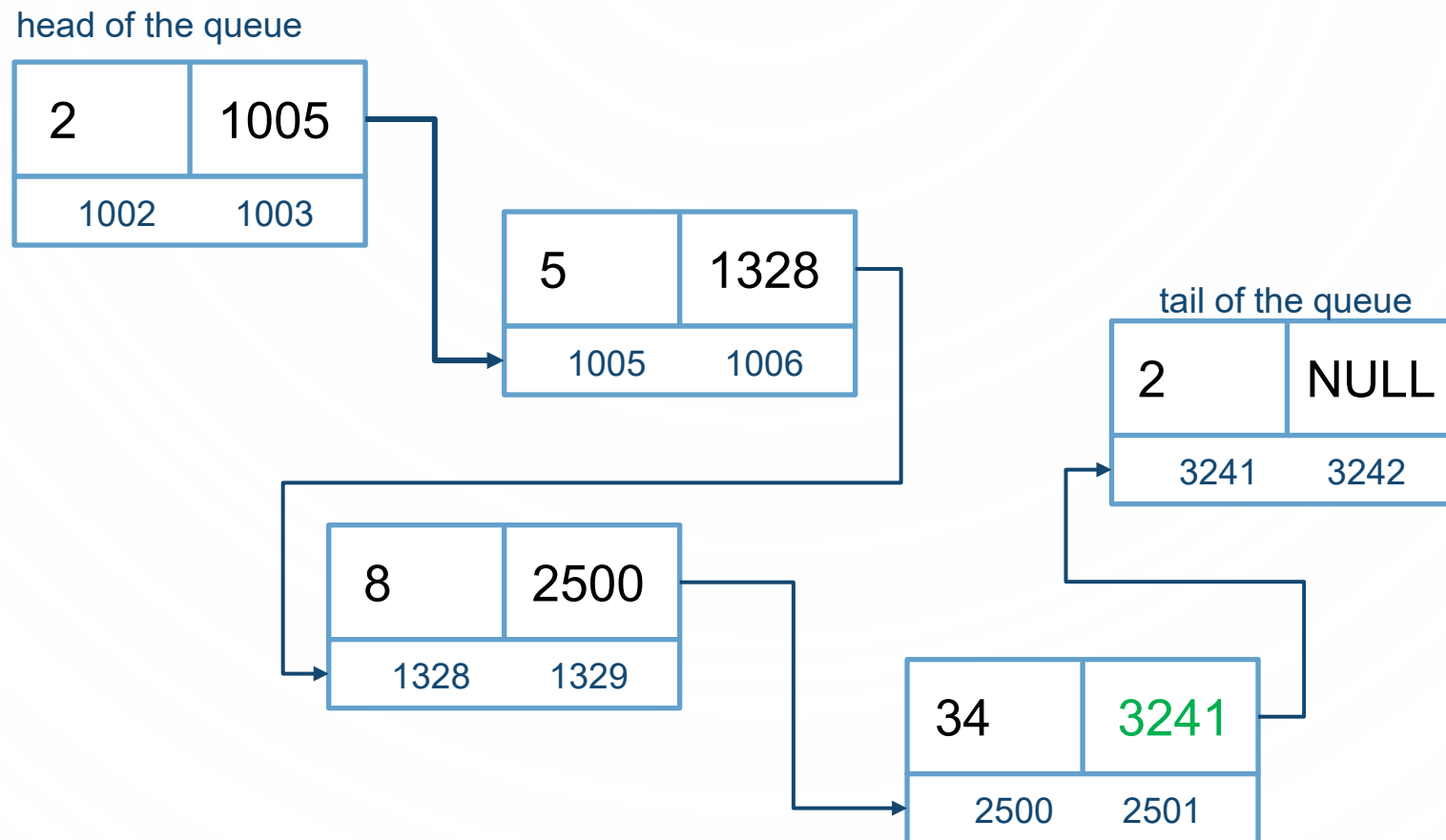
8	2500
1328	1329

tail of the queue

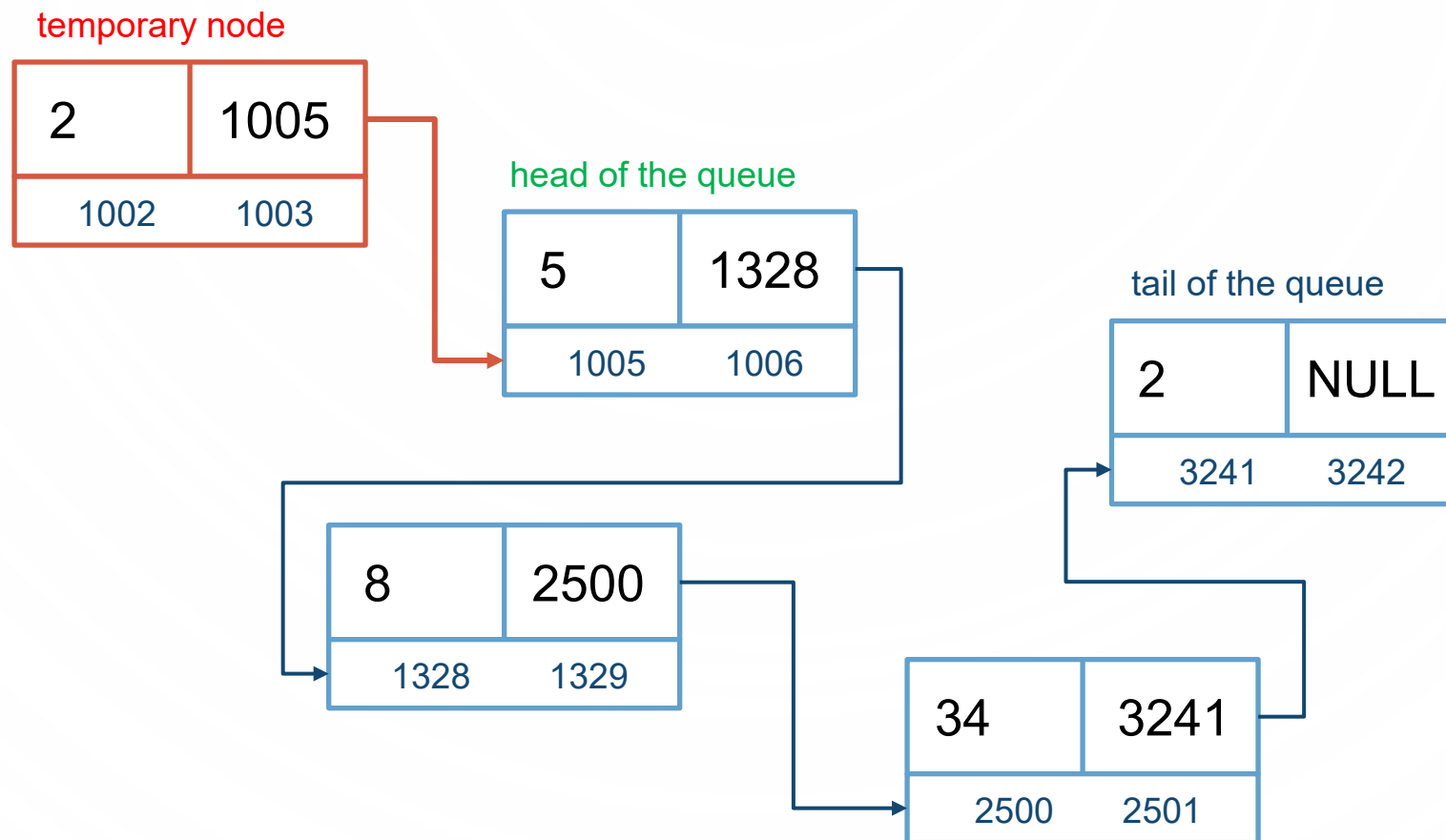
34	NULL
2500	2501



Queues with Linked Lists

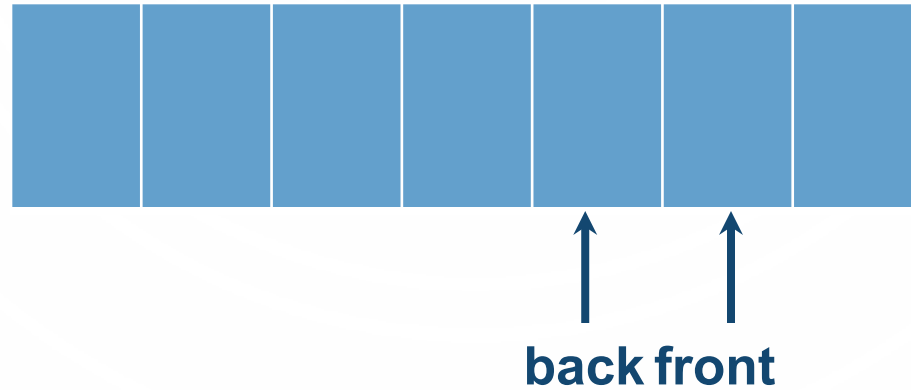


Queues with Linked Lists



Queue – Circular array

Initial state ($\text{back} < \text{front} \ \&\& \ \text{empty} == \text{true}$):



Queue – Circular array

Enqueue (2):

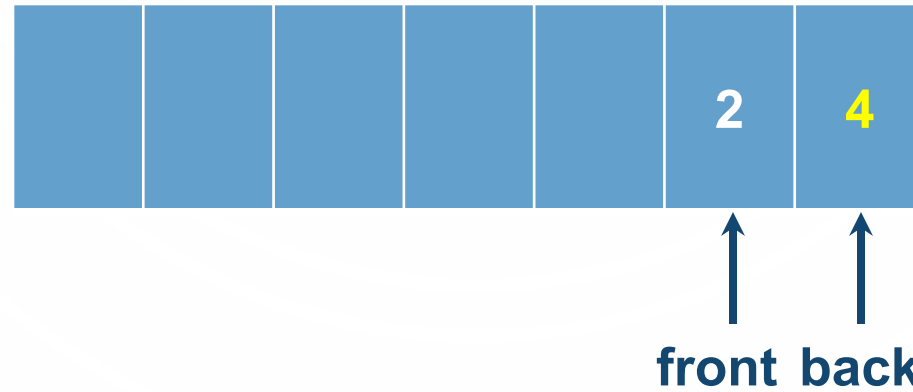


↑
front
back



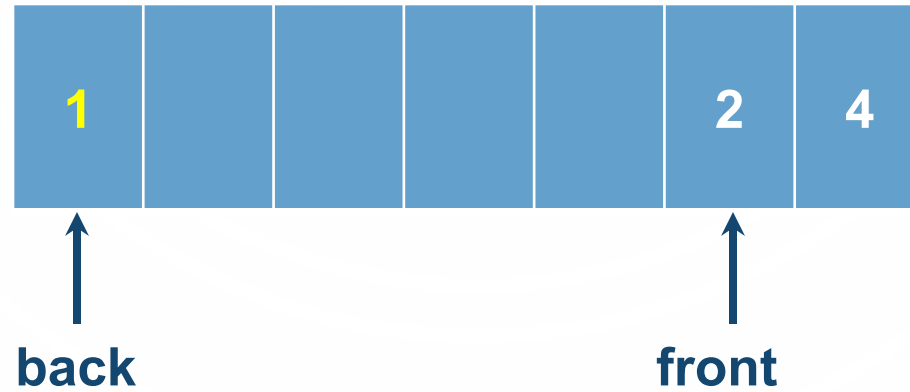
Queue – Circular array

Enqueue (4):



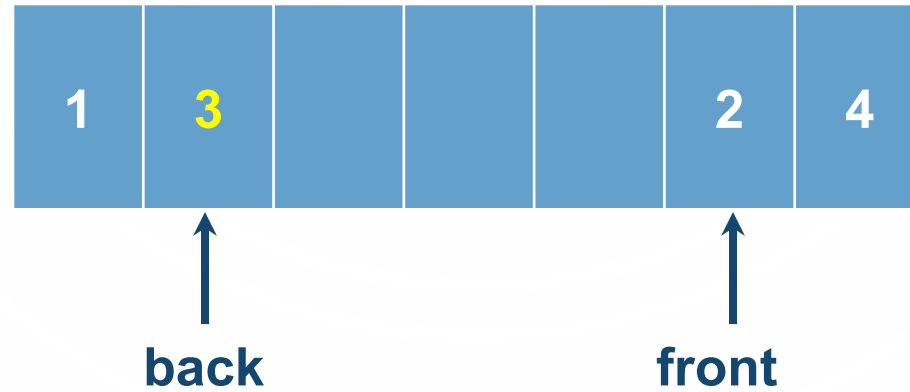
Queue – Circular array

Enqueue (1):



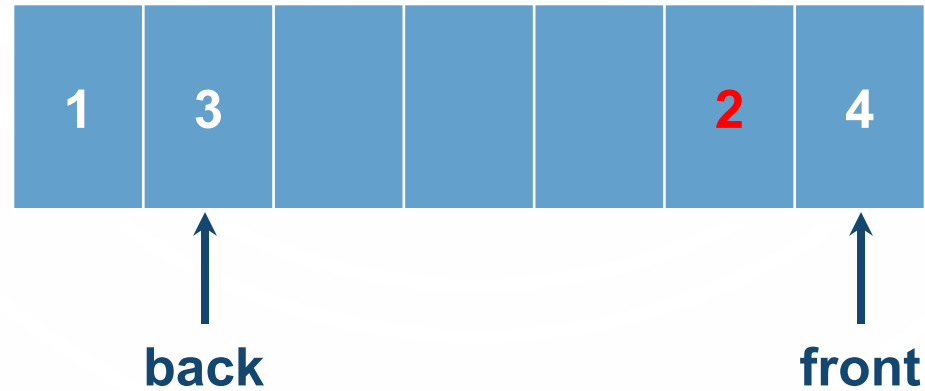
Queue – Circular array

Enqueue (3):



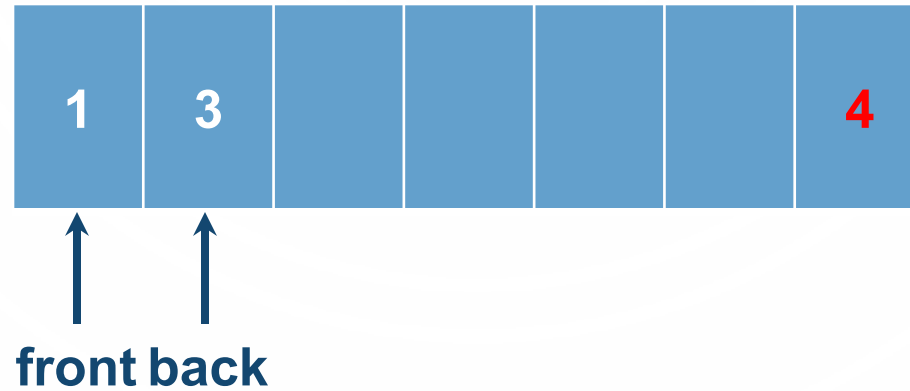
Queue – Circular array

Dequeue => return 2:



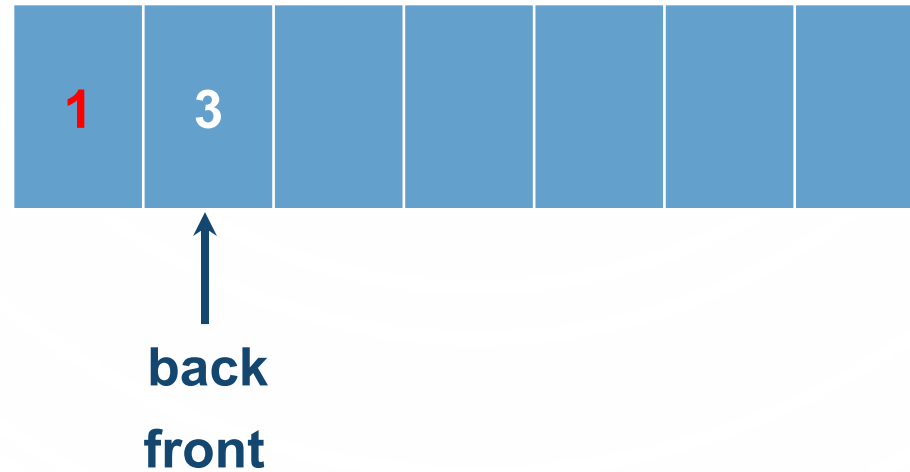
Queue – Circular array

Dequeue => return 4:



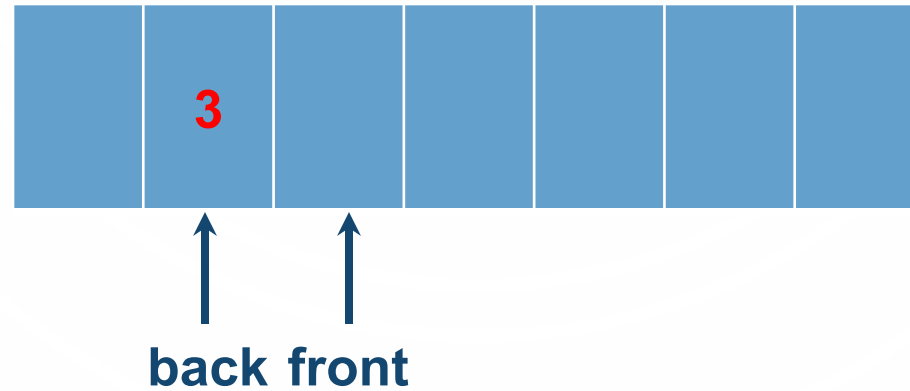
Queue – Circular array

Dequeue => return 1:

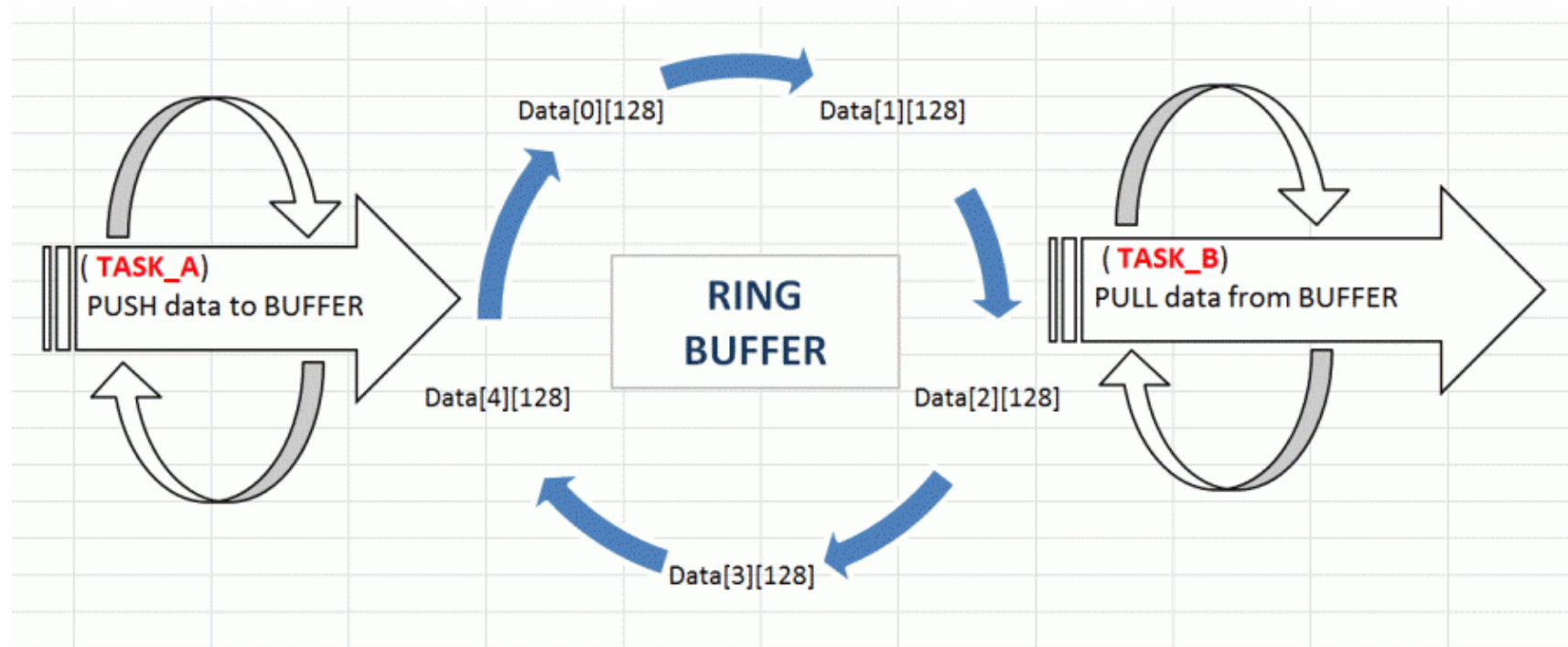


Queue – Circular array

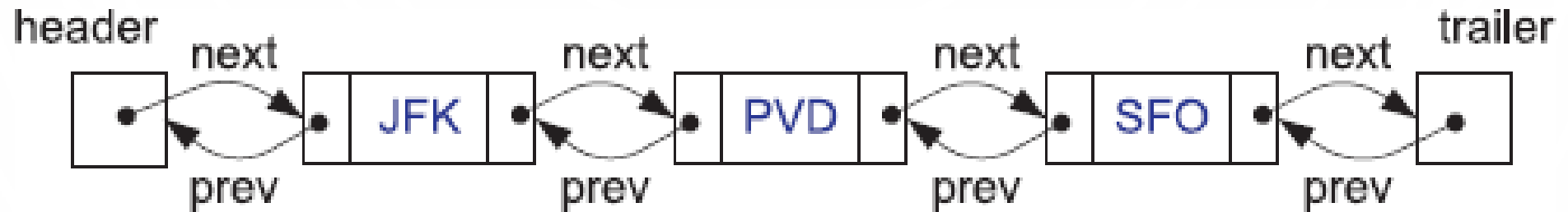
Dequeue => return 3:



Ring buffer



Double Ended Queue – Deque / Deck



Deck Performance

<i>Operation</i>	<i>Time</i>
size	$O(1)$
empty	$O(1)$
front, back	$O(1)$
insertFront, insertBack	$O(1)$
eraseFront, eraseBack	$O(1)$



References

- Ion IVAN, Marius POPA, Paul POCATILU, coordonatori – *Structuri de date*, Editura ASE, Bucuresti, 2008, ISBN 978-606-505-031-0
- Ion SMEUREANU – *Programarea in limbajul C/C++*, Editura CISON, 2001
- Michael MAIN – *Data Structures & Other Objects Using Java*, Editura Pearson, 2012, ISBN: 9780132911504
- <http://www.acs.ase.ro>
- <http://www.itcsolutions.eu>

